Tổng hợp literature Long-CoT theo A1–A4 và Related Work

Ngày 8 tháng 11 năm 2026

Tóm tắt nội dung

Bản thảo phục vụ hai đầu ra: tổng hợp paper Long-CoT theo Research Role A1–A4 và viết Related Work về các survey liên quan. Phần chính trình bày 24 paper theo 16 hướng nhỏ. Mỗi paper có bài toán, cơ chế phương pháp, thiết lập đánh giá và metric, kết quả, đóng góp, hạn chế, nhận xét đối chiếu và vị trí nguồn. Mỗi subgroup có phân tích cơ chế và ranh giới; bốn bảng so sánh tổng hợp phương pháp trong A1–A4. Related Work đối chiếu chín survey theo phạm vi, đơn vị phân loại, cách sử dụng và giới hạn diễn giải. Paper liên quan nhiều nhóm được dẫn chéo thay vì đếm lại. Kết quả là tác giả báo cáo, chưa được nhóm tái lập; nhận xét so sánh là tổng hợp của nhóm.

Mục lục

1 Literature Review theo Research Role **2**
- 1.1 A1: Training/Data . . . 2
- 1.2 A2: Inference/Control . . . 6
- 1.3 A3: Analysis . . . 10
- 1.4 A4: Evaluation/Benchmark . . . 14

2 Related Work: phạm vi và cách phân loại của các survey **19**
- 2.1 Các survey tiêu biểu . . . 19
- 2.2 Đối chiếu các trục phân loại . . . 22
- 2.3 Tổng hợp liên hệ với bản review A1–A4 . . . 22

1 Literature Review theo Research Role

Danh mục chọn lọc gồm 24 paper (A1: 6, A2: 5, A3: 8, A4: 5): 19 nghiên cứu trực tiếp Long-CoT, bốn nghiên cứu liên quan và một công trình nền tảng. Mỗi paper có một primary role; dẫn chéo không tăng số lượng. Kết quả là tác giả báo cáo trong thiết lập được nêu, chưa được nhóm tái lập.

Bản chi tiết giữ bốn trường chính: bài toán, phương pháp, kết quả, đóng góp; bổ sung thiết lập/metric, hạn chế, nhận xét đối chiếu và vị trí nguồn cho từng paper. Nhận xét đối chiếu là tổng hợp của nhóm, không phải kết quả thực nghiệm mới. Mỗi subgroup giải thích cơ chế chung và ranh giới so sánh; cuối A1–A4 có bảng đối chiếu phương pháp. RQ vẫn ở tài liệu bổ sung.

Liên hệ dependency chỉ được ghi khi có bằng chứng; B=N/A nghĩa là chưa gán chức năng dependency, không loại paper khỏi phạm vi Long-CoT.

1.1 A1: Training/Data

Làm thế nào học và chuyển giao năng lực Long-CoT từ dữ liệu và feedback?

A1.1 — RL hình thành năng lực reasoning

Học policy từ reward độ đúng, mở rộng rollout/context và khuyến khích exploration, self-verification.

Cơ chế và cách tiếp cận. RL đặt feedback về độ đúng vào vòng rollout–update, cho phép policy điều chỉnh cách phân bổ reasoning. R1-Zero khảo sát bỏ SFT khởi động; R1 và Kimi còn có các thành phần chuẩn bị dữ liệu, reward hoặc training nhiều stage. Vì vậy cần mô tả recipe đầy đủ, thay vì gộp mọi kết quả dưới nhãn RL.

Đối chiếu và ranh giới. Đối chiếu trung tâm là năng lực học được từ feedback và điều kiện verifier/base. Trace dài hơn là hành vi quan sát, chưa chứng minh từng bước đều cần thiết; khả năng chuyển giao sang model nhỏ được tách ở A1.5.

DeepSeek-R1 [6]. *Long-CoT trực tiếp.*
DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.

Bài toán. Học năng lực Long-CoT qua RL và chuyển giao sang model nhỏ.

Phương pháp. DeepSeek-R1-Zero áp dụng thuật toán GRPO (không critic network) với reward dựa trên độ đúng đáp án và định dạng tag; DeepSeek-R1 kết hợp quy trình multi-stage (cold-start SFT, reasoning RL, rejection sampling tạo 800k mẫu, general RL) và chưng cất sang Qwen/Llama. Tách hai thí nghiệm: R1-Zero khảo sát RL từ base không có SFT khởi động; R1 là pipeline nhiều giai đoạn để tăng tính hữu dụng. GRPO dùng so sánh reward trong nhóm rollout, không có nghĩa training không cần nhiều mẫu sinh.

Thiết lập và metric. Phân biệt R1-Zero, R1 hoàn chỉnh và distilled model; báo Pass@1 trên AIME24/MATH-500, không trộn với self-consistency. R1 là 671B MoE; kết quả chuyển giao được báo riêng cho Distill-Qwen-32B.

Kết quả. R1-Zero đạt 77.9% AIME24 và 95.9% MATH-500; DeepSeek-R1 (671B MoE) đạt 79.8% AIME24, 97.3% MATH-500, 71.5% GPQA Diamond; Distill-Qwen-32B đạt 72.6% AIME24.

Đóng góp. Cung cấp quy trình RL hình thành self-verification/exploration và chuyển giao reasoning qua SFT.

Hạn chế. Gặp khó khăn trong tác vụ tổng quát, pha trộn ngôn ngữ khi huấn luyện pure RL thuần túy, nhạy cảm với reward hacking nếu thiếu verifier tất định.

Nhận xét đối chiếu. Dùng làm mốc cho capability-oriented RL và distillation. Không suy ra RL đã tạo toàn bộ tri thức từ đầu; base model và verifier vẫn là điều kiện thiết yếu. So sánh phải giữ đúng bản v2 và phân biệt Pass@1 với self-consistency.

Vị trí nguồn: arXiv:2501.12948v2, §2.1–2.3, §3, Table 2, Appendix B.3.3 và F. Không trộn số liệu/đánh số mục với v1. [Bản đã dùng].

Kimi k1.5 [10]. *Long-CoT trực tiếp.*
Kimi k1.5: Scaling Reinforcement Learning with LLMs.
Bài toán. Mở rộng RL cho reasoning dài và giảm chi phí triển khai.
Phương pháp. Warmup SFT kết hợp Online Policy Mirror Descent không dùng value model (value-head free), khai thác gradient âm từ vết giải lỗi, chính quy hóa relative entropy, CoT Reward Model (dùng suy luận CoT để phán quyết độ đúng đáp án cuối có giải trình JSON), và khung nén Long2short. Phân biệt context scaling trong RL với Long2short sau đó. CoT-RM dùng lập luận để chấm độ đúng đáp án cuối; tên gọi này không đồng nghĩa có nhãn reward đúng/sai ở từng bước của policy.
Thiết lập và metric. Đánh giá reasoning văn bản và đa phương thức, gồm AIME24, MATH-500 và MathVista. Accuracy giải bài của policy và accuracy của CoT-RM trên spot-check là hai phép đo khác nhau.
Kết quả. Đạt 77.5% Pass@1 AIME24, 96.2% MATH-500, 74.9% MathVista; CoT-RM phán quyết độ đúng đáp án cuối đạt độ chính xác ~98.5% trên mẫu kiểm tra thủ công của tác giả (so với ~84.4% của Classic RM); Long2short giảm độ trễ và độ dài chuỗi.
Đóng góp. Kết nối RL scaling, thiết kế reward và chuyển giao năng lực sang trace ngắn.
Hạn chế. Tiêu tốn chi phí hạ tầng tính toán lớn; CoT-RM đòi hỏi thêm compute khi chấm điểm so với outcome RM thông thường.
Nhận xét đối chiếu. Dùng để thảo luận chuyển giao năng lực từ long sang short traces. Accuracy của reward model trên mẫu spot-check không phải accuracy giải bài; cần tách compute của policy, rollout và reward model khi so sánh chi phí.
Vị trí nguồn: arXiv:2501.12599v4, §2.2, §2.3.2, §2.3.5, Table 2, Figure 10 [Bản đã dùng].

A1.2 — SFT với dữ liệu được chọn lọc

Tuyển chọn demonstration chất lượng cao để khai thác năng lực reasoning của checkpoint mạnh.
Cơ chế và cách tiếp cận. SFT mô phỏng demonstration; lựa chọn dữ liệu quyết định kiểu reasoning model được nhìn thấy. s1 ưu tiên tập s1K nhỏ nhưng khó/đa dạng/chất lượng; LIMO lọc cả câu hỏi lẫn cấu trúc trace. Bài toán vì thế không chỉ là giảm số mẫu, mà là giữ tín hiệu học có ích trong số mẫu đó.
Đối chiếu và ranh giới. Hai hướng cùng gợi ý elicitation trên checkpoint mạnh, nhưng không tạo phép so sánh có kiểm soát chỉ bằng điểm AIME. Chi phí teacher, filtering và inference intervention phải được ghi riêng; s1 còn có thành phần budget forcing ngoài training.

s1 [19]. *Long-CoT trực tiếp.*
s1: Simple test-time scaling.
Bài toán. Kích hoạt reasoning và test-time scaling với ít dữ liệu SFT.
Phương pháp. Tinh chọn tập s1K (1,000 bài toán khó) với vết suy luận chưng cất từ Gemini 2.0 Flash Thinking Experimental; huấn luyện SFT chuẩn trên Qwen2.5-32B-Instruct; kết hợp kỹ thuật test-time budget forcing (chặn `</thought>`, chèn "Wait"). SFT trên s1K và budget forcing là hai thành phần khác nhau: phần đầu thay trọng số, phần sau ép kéo dài hoặc kết thúc generation. Chất lượng lựa chọn dữ liệu quan trọng hơn việc chỉ báo cáo số mẫu nhỏ.
Thiết lập và metric. Base Qwen2.5-32B-Instruct, SFT trên 1,000 mẫu s1K. So sánh s1-32B với/không budget forcing trên AIME24; các benchmark bổ sung gồm MATH-500 và GPQA Diamond.
Kết quả. s1-32B không can thiệp đạt 50.0% AIME24; khi bật budget forcing đạt 56.7% AIME24, 93.0% MATH-500, 59.6% GPQA Diamond (tăng +30.0 pp AIME so với base 26.7%).
Đóng góp. Cho thấy hiệu quả dữ liệu chọn lọc và cơ chế điều khiển độ dài đơn giản.

Hạn chế. Kết quả phụ thuộc vào base model và chất lượng vết giải từ teacher trong thiết lập đã thử; không chứng minh cơ chế tri thức của mọi mô hình.
Nhận xét đối chiếu. Dùng để nối A1 với A2: cần ablation SFT-only, control-only và kết hợp trên cùng checkpoint. Số mẫu fine-tuning không bao gồm chi phí tạo/lọc dữ liệu teacher; không xem đây là tổng chi phí xây dựng reasoning model.
Vị trí nguồn: arXiv:2501.19393, §2.1, §3.1, §4.1, Table 1 [Bản đã dùng].

LIMO [29]. *Long-CoT trực tiếp.*
LIMO: Less is More for Reasoning.
Bài toán. Giảm lượng demonstration cần để học reasoning toán khó.
Phương pháp. Lọc câu hỏi 2 tầng (loại bài dễ, giữ bài DeepSeek-R1-Distill-32B giải đúng 1-3/32 lần); chấm điểm vết suy luận theo rubric 4 chiều (lập luận, tự kiểm tra, thăm dò, độ hạt thích ứng); huấn luyện SFT trên 817 mẫu chọn lọc nhất với Qwen2.5-32B-Instruct. LIMO lọc cả độ khó câu hỏi lẫn chất lượng vết giải. Một bộ dữ liệu nhỏ nhưng có self-verification và exploration khác với cắt ngẫu nhiên một tập SFT lớn xuống cùng số mẫu.
Thiết lập và metric. SFT Qwen2.5-32B-Instruct trên 817 mẫu, đánh giá AIME24, MATH-500 và AMC23. Thực nghiệm đổi base sang Qwen1.5-32B-Chat kiểm tra mức phụ thuộc checkpoint.
Kết quả. LIMO-32B đạt 63.3% AIME24, 95.6% MATH-500, 96.3% AMC23; khi chuyển sang base model yếu hơn (Qwen1.5-32B-Chat), hiệu năng AIME24 sụt giảm mạnh xuống 9.2% (-54.1 pp).
Đóng góp. Đề xuất cách chọn dữ liệu chất lượng và chỉ ra điều kiện phụ thuộc năng lực base.
Hạn chế. Hiệu quả mẫu phụ thuộc vào năng lực của base model được thử nghiệm; thực nghiệm không cung cấp bằng chứng trực tiếp về cơ chế tiền huấn luyện.
Nhận xét đối chiếu. Dùng để đối chiếu giả thuyết elicitation trên base mạnh với việc học năng lực mới. Thí nghiệm đổi base làm thay đổi nhiều yếu tố đồng thời; chưa cô lập được tri thức pretraining, instruction tuning và kiến trúc.
Vị trí nguồn: arXiv:2502.03387v3, §3, §3.1, §6.2, Table 1, §6.3.3 [Bản đã dùng].

A1.3 — Data synthesis/selection và curriculum

Sinh hoặc lọc trace, kiểm tra contamination và sắp xếp dữ liệu qua các stage training.
Cơ chế và cách tiếp cận. Hướng này phân tích vòng đời dữ liệu: nguồn bài toán, cách sinh trace, cách giữ lời giải đúng/chất lượng và cách chia stage. Light-R1 cho ví dụ kết hợp dữ liệu đã lọc với curriculum; s1/LIMO làm rõ selection, còn R1 dùng rejection sampling để tạo dữ liệu cho stage sau.
Đối chiếu và ranh giới. Số liệu qua các checkpoint cho biết tiến triển pipeline nhưng chưa tự cô lập đóng góp của từng thành phần. Data quantity, độ khó, teacher và training budget có thể thay đổi cùng lúc; decontamination cần đi cùng claim về hiệu quả dữ liệu.

Light-R1 [26]. *Long-CoT trực tiếp.*
Light-R1: Curriculum SFT, DPO and RL for Long COT from Scratch and Beyond.
Bài toán. Xây quy trình dữ liệu và training Long-CoT mở, có kiểm tra contamination.
Phương pháp. Quy trình gồm 3 giai đoạn huấn luyện (SFT-1 với 76k bài toán đã lọc và độ dài tối đa 20k tokens, SFT-2 với 3k bài thi đấu, DPO bán on-policy) kết hợp Model Merging trên checkpoint Qwen2.5-32B-Instruct; kèm kiểm định khử nhiễm benchmark nghiêm ngặt. Phân tích pipeline theo từng bước SFT-1, SFT-2, DPO và merging; chữ from scratch ở đây không phải huấn luyện từ trọng số ngẫu nhiên. Kiểm tra decontamination là một phần của bằng chứng về chất lượng dữ liệu.
Thiết lập và metric. Khởi tạo Qwen2.5-32B-Instruct; theo dõi AIME24 và GPQA qua SFT-1, SFT-2, DPO và merging. Đây là so sánh qua stage của pipeline, không phải pretraining từ trọng số ngẫu nhiên.

Kết quả. Light-R1-32B tăng tiến AIME24 qua các giai đoạn (Base 16.6% -> SFT-1 69.0% -> SFT-2 73.0% -> DPO 75.8% -> Merging 76.6%), trong khi GPQA biến động (64.3% ở SFT-1 -> 60.6% ở SFT-2 -> 61.8% ở mô hình cuối); phát hiện rò rỉ đề thi trong nhiều tập dữ liệu mở trước đó.
Đóng góp. Công bố pipeline nhiều stage và kiểm tra test-set leakage trong dữ liệu mở.
Hạn chế. Khởi tạo từ checkpoint instruct mạnh đã qua căn chỉnh; phụ thuộc vào dữ liệu thi đấu chất lượng cao ở Stage 2.
Nhận xét đối chiếu. Dùng để thảo luận curriculum và trade-off giữa toán với miền khác: AIME tăng qua các stage nhưng GPQA không tăng đơn điệu. Không quy toàn bộ gain cho một stage nếu không kiểm soát thay đổi dữ liệu, budget và checkpoint.
Vị trí nguồn: arXiv:2503.10460v4, §3.1.2, §3.2, Table 2, Table 7 [Bản đã dùng].

A1.4 — RL hướng efficiency

Học đánh đổi accuracy–length, phân bổ reasoning budget theo độ khó.
Cơ chế và cách tiếp cận. Khác với capability-oriented RL, efficiency-aware training đưa chi phí reasoning vào objective. Model được khuyến khích phân biệt những bài cần deliberation dài với bài có thể kết luận sớm, thay vì áp một độ dài cố định cho mọi input.
Đối chiếu và ranh giới. Đối chiếu cần theo accuracy–cost frontier. Cùng hệ số length penalty có thể nén khác nhau giữa tác vụ; giảm token có thể đổi accuracy, hành vi verification và faithfulness. Vì vậy không mô tả đây là thay đổi chỉ có lợi về compute.

Training Language Models to Reason Efficiently [1]. *Long-CoT trực tiếp.*
Training Language Models to Reason Efficiently.
Bài toán. Học phân bổ compute theo độ khó thay vì luôn sinh trace dài.
Phương pháp. RL với reward kết hợp độ đúng và phạt độ dài, điều chỉnh bằng hệ số alpha. Length penalty thay đổi mục tiêu tối ưu của policy, không chỉ đặt max_tokens khi decode. Cùng một alpha có thể tạo mức nén khác nhau giữa tác vụ dễ và khó.
Thiết lập và metric. Training trên reasoning model DeepSeek-R1-Distill-Qwen ở các mức 1.5B/7B; thay alpha để khảo sát trade-off. GSM8K, MATH500 và AIME24 được dùng để so task difficulty; tiết kiệm token không ngụ ý cùng accuracy.
Kết quả. Với DeepSeek-R1-Distill-Qwen-7B, alpha=0.2 giảm token 83% trên GSM8K nhưng 27% trên AIME24; đây là hai mức tiết kiệm theo tác vụ, không phải cùng độ chính xác.
Đóng góp. Xây các điểm đánh đổi accuracy–cost bằng objective training.
Hạn chế. Nén có thể giảm accuracy và faithfulness; bảo đảm lý thuyết cần các giả định lý tưởng.
Nhận xét đối chiếu. Dùng làm đại diện learned compute allocation. Cần báo accuracy–cost frontier và faithfulness cùng nhau; mệnh đề tối ưu ở population level không bảo đảm một lần huấn luyện hữu hạn giữ nguyên độ đúng.
Vị trí nguồn: v4, §4, §5.3.1–5.3.2, Figure 3 [Bản đã dùng].

A1.5 — Distillation và Long2short

Chuyển năng lực từ teacher hoặc trace dài sang model nhỏ hay trace ngắn; không đếm lại paper.
Cơ chế và cách tiếp cận. Cần tách hai mục tiêu distillation: chuyển năng lực teacher sang model ít tham số hơn và chuyển long reasoning sang trace ngắn hơn. DeepSeek-R1 cung cấp ví dụ trace distillation qua SFT; Kimi Long2short khảo sát chuyển giao từ long-CoT sang short-CoT. Hai mục tiêu có thể cùng xuất hiện nhưng không đồng nghĩa.
Đối chiếu và ranh giới. Khi đối chiếu phải ghi cả teacher/student, model size, cách tạo/chọn trace và evaluation budget. Distillation có thể giảm chi phí student nhưng vẫn cần compute của

teacher khi xây dữ liệu; trace được nén không tự chứng minh đã giữ toàn bộ dependency hữu ích.

Bảng đối chiếu trong A1. *Nhận xét tổng hợp, không phải xếp hạng có kiểm soát.*

Hướng/paper	Thành phần hoặc đơn vị chính	Điểm cần đọc kèm
RL capability	R1, Kimi: thay policy qua rollout và reward	Base, verifier và training recipe quyết định điều kiện kết quả.
SFT/selection	s1, LIMO: thay demonstration và trọng số SFT	Ít mẫu không bao gồm toàn bộ chi phí teacher/filtering.
Dữ liệu/curriculum	Light-R1: thay dữ liệu và thứ tự stage	Theo dõi gain theo miền; kiểm soát contamination và budget.
RL efficiency	Arora/Zanette: thêm length-aware objective	So accuracy–cost frontier, không chỉ tỷ lệ token giảm.
Distillation	R1/Kimi: chuyển tri thức sang student hoặc trace ngắn	Tách model size, trace length và chi phí xây dữ liệu.

Tổng hợp hướng nghiên cứu và hạn chế chung. Các công trình cho thấy có nhiều con đường đến Long-CoT: RL từ feedback, SFT dữ liệu chọn lọc, pipeline nhiều stage và distillation. Tuy nhiên, ít mẫu SFT không có nghĩa ít tổng chi phí; năng lực base, chi phí teacher và verifier vẫn quan trọng. Số điểm giữa các paper chỉ có ý nghĩa khi đối chiếu cùng model, task và budget.

Liên hệ dependency khi có bằng chứng. Với SFT/RL/distillation nói chung, ghi B=N/A nếu chưa có bằng chứng can thiệp trực tiếp vào reasoning dependency. Sinh trace dài hoặc đạt accuracy cao chưa đủ để gán chức năng formation/state carrying; liên hệ trên đây chủ yếu là vai trò training/data.

1.2 A2: Inference/Control

Làm thế nào phân bổ test-time compute và điều khiển trajectory khi model đã được training?

A2.1 — Adaptive reasoning và early stopping

Chọn thời điểm kiểm tra và quyết định khi nào reasoning đã đủ hoặc đang lặp thừa.
Cơ chế và cách tiếp cận. Early stopping gồm ba phần riêng: trigger kiểm tra, tín hiệu ra quyết định và action kết thúc thinking. DTSR tập trung sufficiency; REFRAIN kết hợp redundancy với threshold thích ứng. Chỉ việc hai phương pháp cùng theo dõi token phản tư không khiến tiêu chuẩn dừng giống nhau.
Đối chiếu và ranh giới. So sánh cần phân tích cả trường hợp dừng sớm làm mất đáp án đúng và thời gian/compute bỏ ra cho checking. Chênh lệch accuracy trung bình nhỏ không tự chứng minh equivalence; proxy tự tin hoặc similarity có thể sai trên bài khó.

DTSR — When Is Thinking Enough? [27]. *Long-CoT trực tiếp.*
When Is Thinking Enough? Early Exit via Sufficiency Assessment for Efficient Reasoning.
Bài toán. Dừng reasoning khi đã đủ cơ sở kết luận.
Phương pháp. Cơ chế DTSR không cần huấn luyện, giám sát từ khóa phản tư ("Wait", "Alternative") để xác định điểm kiểm tra, sau đó dùng prompt tự đánh giá độ đủ suy luận (thought sufficiency); nếu vượt ngưỡng tau thì ép dừng `</think>`. Tách thời điểm kiểm tra khỏi tiêu chuẩn dừng: tín hiệu phản tư/chu kỳ kiểm tra quyết định khi nào hỏi, sufficiency assessment quyết định có đủ để kết luận hay chưa.
Thiết lập và metric. So với vanilla reasoning trên các cỡ Qwen3; ghi riêng accuracy và số token. Kết quả Qwen3-14B và Qwen3-8B cho thấy tỷ lệ tiết kiệm và accuracy có thể thay đổi khác nhau theo model size.

Kết quả. Trên Qwen3-14B, độ chính xác đạt 84.8% (+0.4 pp so với baseline 84.4%) trong khi giảm 34.9% token; trên Qwen3-8B, token giảm 32.0% nhưng độ chính xác giảm nhẹ 0.9 pp (81.0% vs 81.9%).
Đóng góp. Đưa sufficiency vào early stopping training-free.
Hạn chế. Hiệu năng tự đánh giá độ đủ suy luận phụ thuộc vào quy mô mô hình; mô hình nhỏ hơn có thể dừng sớm khi chưa giải quyết trọn vẹn bài toán.
Nhận xét đối chiếu. Dùng để so với fixed-budget truncation và confidence stopping. Accuracy trung bình gần nhau chưa chứng minh từng bài đều được giữ đúng; cần phân tích correct→wrong khi dừng sớm và đo latency gồm các lần sufficiency check.
Vị trí nguồn: arXiv:2604.06787v1, §2.2, §3.2, Table 1 [Bản đã dùng].

REFRAIN — Stop When Enough [23]. *Long-CoT trực tiếp.*
Stop When Enough: Adaptive Early-Stopping for Chain-of-Thought Reasoning.
Bài toán. Giảm reflection lặp thừa và thích ứng ngưỡng dừng.
Phương pháp. REFRAIN kiểm tra đồng thời giả thuyết đáp án, token phản tư và độ dư thừa ngữ nghĩa cosine similarity giữa embedding câu hiện tại và quá khứ; dùng thuật toán Sliding-Window UCB bandit để tự điều chỉnh ngưỡng dừng tau. REFRAIN kết hợp đáp án hiện có, trigger phản tư và similarity; SW-UCB cập nhật ngưỡng từ reward gồm answer-only likelihood và phạt độ dài. Tín hiệu quality này là proxy không cần ground-truth test.
Thiết lập và metric. Đánh giá trên reasoning model như gpt-oss-20B và Qwen3-8B với GSM8K, MATH-500, GPQA Diamond. Đo Pass@1 và token consumption; reward online của controller dựa trên answer-only likelihood và chi phí độ dài.
Kết quả. Trên gpt-oss-20B, tăng Pass@1 trên toàn bộ benchmark thử nghiệm (+2.73 pp GSM8K, +3.40 pp MATH-500, +7.07 pp GPQA Diamond) và giảm 33.3% đến 44.7% token; trên Qwen3-8B tiết kiệm 21.5% đến 54.2% token.
Đóng góp. Kết hợp redundancy detection với online stopping control.
Hạn chế. Dư thừa phản tư dựa trên embedding ngữ nghĩa không đồng nhất hoàn toàn với độ đủ logic hình thức; phụ thuộc vào chất lượng mô hình embedding câu.
Nhận xét đối chiếu. Dùng để phân biệt online adaptation của controller với cập nhật trọng số LLM. Similarity có thể nhầm phép kiểm tra hữu ích với lặp thừa; likelihood cao cũng không bảo đảm correctness. Xem §3.3–3.6 cho cơ chế reward.
Vị trí nguồn: arXiv:2510.10103v1, §3.2, §3.4, §5.1, Table 1 [Bản đã dùng].

A2.2 — Search, branching và phục hồi trajectory

Tạo/chọn nhánh bằng feedback hoặc self-evaluation; prune, backtrack hoặc restart khi phát hiện bế tắc.
Cơ chế và cách tiếp cận. MCTSr tạo search tree trên các lời giải hoàn chỉnh rồi critique/refine; TAAR tập trung phát hiện prefix mắc trap để cắt và restart. Cả hai thay trajectory bằng feedback, nhưng đơn vị search và lý do tạo nhánh mới khác nhau.
Đối chiếu và ranh giới. Verification-guided selection ở cấp solution/trajectory cần phân biệt với token-level decoding. Ngân sách K hoặc rollout bằng nhau chưa khớp tổng cost: còn evaluation, rewrite, prefix bị bỏ và các nhánh thất bại. Policy chẩn đoán được training trước cũng khác một controller hoàn toàn training-free.

TAAR — Thinking Traps [3]. *Long-CoT trực tiếp.*
Thinking Traps in Long Chain-of-Thought: A Measurable Study and Trap-Aware Adaptive Restart.
Bài toán. Thoát prefix mắc bẫy thay vì tiếp tục tự sửa trên tiền đề sai.
Phương pháp. Khung TAAR gồm policy chẩn đoán nhẹ pi_phi dự đoán vị trí bẫy t_hat và xác suất thoát lỗi p_hat; bộ điều khiển thực hiện Cut@Trap cắt bỏ phân đoạn bẫy và tái khởi động suy luận với nhiệt độ thích ứng. Đơn vị can thiệp là prefix được chẩn đoán mắc bẫy: cắt

trước đoạn lỗi rồi sinh lại. Khác với tự phản tư chỉ nối thêm token vào cùng prefix sai, nhưng cần policy chẩn đoán đã huấn luyện.
Thiết lập và metric. So Avg@4, PRM@4 và TAAR với K=4 trên R1-Distill-Qwen-8B, lấy trung bình qua năm benchmark thi đấu. Có đối chiếu GPT-OSS-120B; path budget được giữ không có nghĩa mọi method dùng cùng tổng token.
Kết quả. Với cùng ngân sách đường đi báo cáo K=4, trên DeepSeek-R1-Distill-Qwen-8B, độ chính xác trung bình trên 5 benchmark đạt 69.1% (tăng +4.6 pp so với Avg@4 64.5% và +3.1 pp so với PRM@4 66.0%); Cut@Trap đạt escape rate 16.5% (8B); trên GPT-OSS-120B đạt 79.2% (thấp hơn Avg@4 80.6%).
Đóng góp. Nối chẩn đoán thinking trap với action phục hồi lúc inference.
Hạn chế. Không đo trực tiếp cơ chế chú ý (attention mechanism); trên mô hình biên giới rất lớn (120B) có thể cắt nhầm đường dẫn gần đúng làm giảm độ chính xác.
Nhận xét đối chiếu. Dùng để nối phân tích lỗi A3 với recovery A2. Cùng K=4 chưa đồng nghĩa cùng tổng token hoặc thời gian; phải tính phần prefix bị bỏ, lượt chẩn đoán và rollout sinh lại. Kết quả trên 120B cho thấy restart không luôn có lợi.
Vị trí nguồn: arXiv:2601.11940v1, §3.1, Figure 4, §5.2, Table 2 [Bản đã dùng].

MCTSr [30]. *Nghiên cứu suy luận liên quan.*
Accessing GPT-4 level Mathematical Olympiad Solutions via Monte Carlo Tree Self-refine with LLaMa-3 8B.
Bài toán. Tăng chất lượng reasoning toán bằng search và self-refinement.
Phương pháp. Thuật toán MCTSr kết hợp tìm kiếm cây Monte Carlo với self-refine văn bản: mỗi nút là một lời giải hoàn chỉnh, mở rộng bằng critique và viết lại cải tiến, tự chấm điểm (-100 đến 100) và lan truyền ngược cập nhật giá trị. Nút search là một lời giải hoàn chỉnh được refine; số rollout đếm các lần mở rộng/tinh chỉnh chứ không phải độ dài một chain. Self-evaluation là nguồn tín hiệu chọn nhánh.
Thiết lập và metric. LLaMA-3-8B không fine-tune, so zero-shot với 8 rollout trên MATH; paper cũng báo Math Odyssey. Metric là accuracy giải bài sau search, không phải accuracy của một bước hay một verifier riêng.
Kết quả. Trên LLaMA-3-8B không fine-tune, MATH accuracy tăng từ 24.36% (zero-shot) lên 58.24% (8-rollouts); Math Odyssey đạt 49.36% (ngang ngửa GPT-4 Turbo 49.1%).
Đóng góp. Cung cấp đối chiếu về branching/search và feedback-guided lựa chọn lời giải.
Hạn chế. Tìm kiếm ở cấp độ cả lời giải (coarse granularity) tiêu tốn nhiều lượt sinh văn bản hoàn chỉnh; điểm số tự đánh giá có thể thiếu tin cậy ở bài toán quá khó. Scope thuộc adjacent_reasoning.
Nhận xét đối chiếu. Dùng làm đối chiếu search bên ngoài cho Long-CoT nội sinh. Gain trên LLaMA-3-8B không chứng minh cùng mức tiết kiệm với early exit; cần cộng token cho critique, rewrite, evaluation và mọi nhánh search.
Vị trí nguồn: arXiv:2406.07394v2, §3, §4.3, Table 2, §4.4, Table 3 [Bản đã dùng].

A2.3 — Cô lập context và quản lý state

Dùng call/return và local context để tổ chức subproblem và chuyển intermediate result.
Cơ chế và cách tiếp cận. Recursive execution thay đổi nơi state được giữ: local context xử lý subproblem, return value mang thông tin về parent. Cách này không chỉ cắt bớt văn bản mà còn thay giao diện trao đổi intermediate result.
Đối chiếu và ranh giới. Cần phân biệt peak active context, tổng token và độ đúng return state. Lợi ích trên tác vụ có decomposition rõ không tự chuyển sang mọi reasoning task; so sánh OOD theo độ dài đặc biệt hữu ích để đánh giá cách quản lý state.

Recursive Models [28]. *Long-CoT trực tiếp.*
Recursive Models for Long-Horizon Reasoning.

Bài toán. Quản lý state/context khi reasoning kéo dài.
Phương pháp. Mô hình đệ quy (f‾rm) phân rã bài toán thành các stack frame độc lập qua thẻ `<call>` và `<return>`; khi bài toán con hoàn thành chỉ trả về kết quả ngắn gọn b cho ngữ cảnh cha và giải phóng trace trung gian. Mô hình đệ quy thay đổi cách mang trạng thái: mỗi call có local context, chỉ return kết quả cho parent. Rút gọn context hoạt động khác với việc toàn bộ quá trình dùng ít token hơn.
Thiết lập và metric. Fine-tune Qwen2.5-3B-Instruct cho thực thi đệ quy. SAT được tách Easy/Medium/Hard OOD; Go 4x4 dùng length-OOD và trace accuracy, khác với chỉ chấm đáp án cuối.
Kết quả. Fine-tune Qwen2.5-3B-Instruct đệ quy trên SAT solving đạt 98% (Easy), 95% (Medium), 64% (Hard OOD), vượt GPT-4o phẳng (48.8% Hard); trên 4x4 Go (Length-OOD) đạt 38.5% trace accuracy trong khi CoT phẳng sụt giảm xuống 1.0%.
Đóng góp. Đề xuất thực thi đệ quy giúp cô lập context và giữ kết quả subproblem.
Hạn chế. Đòi hỏi định dạng bài toán có thể phân rã cấu trúc đệ quy rõ ràng; chi phí quản lý stack và chuyển ngữ cảnh ngoài bộ nhớ.
Nhận xét đối chiếu. Dùng để mở rộng A2 từ stopping sang memory/state management. So sánh cần cả tổng token, peak context và chi phí stack; bài toán phải có decomposition hợp lệ. Không đồng nhất framework đã fine-tune với prompting đệ quy cho model bất kỳ.
Vị trí nguồn: arXiv:2603.02112v2, §3, §5.1, Table 1, Table 2 [Bản đã dùng].

A2.4 — Test-time compute và điều khiển độ sâu

Điều tiết độ dài, thời điểm đổi hướng hoặc selection/pruning; giữ primary role của paper gốc.
Cơ chế và cách tiếp cận. Test-time control có thể tác động lên nhiều tài nguyên: s1 điều tiết thời lượng bằng budget forcing; TIP điều tiết thời điểm đổi hướng; FSF hỗ trợ chọn/prune theo cấu trúc. Đây là các paper dẫn chéo từ A1/A3, không phải bản ghi bổ sung.
Đối chiếu và ranh giới. Tổng độ dài, số candidate và độ sâu exploration cần báo riêng. Ép dài hơn có thể thêm kiểm chứng hoặc lặp thừa; ép ngắn hơn có thể bỏ obligation chưa giải quyết. Kết quả dưới một budget không thay thế đường cong accuracy–cost.

Bảng đối chiếu trong A2. *Nhận xét tổng hợp, không phải xếp hạng có kiểm soát.*

Hướng/paper	Thành phần hoặc đơn vị chính	Điểm cần đọc kèm
DTSR	Đánh giá sufficiency rồi đóng thinking	Dừng sớm sai và overhead checking.
REFRAIN	Redundancy detector và threshold thích ứng	Similarity/likelihood không bảo đảm correctness.
MCTSr	Chọn/refine lời giải trong search tree	Chi phí critique, evaluation và nhánh thất bại.
TAAR	Chẩn đoán trap, cắt prefix và restart	Cắt nhầm prefix gần đúng; diagnostic policy có training.
Recursive Models	Tách local context và trả state	Peak context và tổng execution cost là hai phép đo.
s1/TIP/FSF	Điều tiết thời lượng, switching hoặc selection	Giữ primary role gốc và tính toàn bộ candidate/judge cost.

Tổng hợp hướng nghiên cứu và hạn chế chung. Các phương pháp khác nhau ở tín hiệu quan sát và action: kéo dài, dừng, chọn nhánh, sửa prefix, restart hoặc tách context. Search/branching và feedback-guided selection được thể hiện ở MCTSr; TAAR so với PRM@4 và dùng chẩn đoán để restart. Đây là bằng chứng ở mức trajectory/solution, không tự đồng nghĩa verification-guided decoding ở từng token. So sánh cần tính cả evaluator call, nhánh bị bỏ và overhead kiểm tra.

Liên hệ dependency khi có bằng chứng. Có liên hệ theo chức năng khi thiết kế thể hiện rõ: Recursive Models mang state giữa các subproblem; TAAR sửa/reconfigure trajectory; DTSR đánh giá sufficiency để đóng reasoning. Budget forcing vì đạt giới hạn token chưa tự chứng minh closure của một reasoning obligation.

1.3 A3: Analysis

Long reasoning có những hành vi, failure mode và giới hạn nào; bằng chứng hỗ trợ đến đâu?

A3.1 — Cấu trúc diễn ngôn và interpretability/faithfulness

Phân biệt trace nhìn thấy được với dependency của tính toán nội tại; kiểm tra bằng perturbation.
Cơ chế và cách tiếp cận. Discourse analysis hỏi các đoạn trace liên hệ thế nào; faithfulness hỏi trace có phản ánh yếu tố ảnh hưởng đáp án hay không. ReasoningFlow và Lanham cung cấp hai kiểu công cụ: schema/graph và perturbation trên reasoning hoặc tính toán liên quan.
Đối chiếu và ranh giới. Không đồng nhất interpretability của biểu diễn với chứng minh mechanism. Attention masking, chèn lỗi hay early answering đều cần được đọc theo phép can thiệp, task và model cụ thể; distribution shift có thể ảnh hưởng cách diễn giải.

ReasoningFlow [13]. *Long-CoT trực tiếp.*
ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces.
Bài toán. Mô tả discourse dependency và kiểm tra liên hệ với tính toán nội tại.
Phương pháp. Đề xuất ReasoningFlow biểu diễn vết suy luận thành DAG với các nhãn quan hệ diễn ngôn (support, attack, verify, assumption); thực hiện can thiệp Thought Anchors (attention masking) để đo điểm phụ thuộc nhân quả thực nghiệm qua log KL-divergence. Schema discourse mô tả quan hệ giữa các bước ở mức văn bản. Thí nghiệm Thought Anchors kiểm tra một câu hỏi khác: thao tác attention có thay đổi phân phối dự đoán bước sau hay không.
Thiết lập và metric. Đối chiếu discourse edge với điểm Thought Anchors trong setup QwQ/AIME; đánh giá alignment bằng precision-recall. Các tỷ lệ bước sửa đúng và đáp án cuối đúng là thống kê hành vi trên trace.
Kết quả. Trên QwQ và AIME, điểm phụ thuộc Thought Anchors khi dự đoán cạnh ReasoningFlow không vượt trội so với baseline heuristic chọn nút gần nhất theo khoảng cách token; trên quan sát vết giải, bước kiểm tra cục bộ có sửa lỗi ghi nhận tỷ lệ bước đúng là 84% (so với 64%) và tỷ lệ dẫn đến đáp án đúng cuối là 20% (so với 5%).
Đóng góp. Cung cấp schema trace và bằng chứng phân biệt discourse structure với mechanistic dependency.
Hạn chế. Tỷ lệ đúng của bước sửa lỗi là quan sát tương quan hành vi trên tập vết giải, không phải can thiệp nhân quả có đối chứng độc lập; chú ý Thought Anchors chỉ kiểm tra trên mô hình và tác vụ cụ thể.
Nhận xét đối chiếu. Dùng để phân biệt representation và mechanism. Cạnh diễn ngôn, điểm can thiệp và tỷ lệ sửa đúng là ba loại bằng chứng khác nhau; alignment yếu trong một setup không phủ nhận mọi phương pháp mechanistic analysis.
Vị trí nguồn: arXiv:2606.05402v2, §3, §6.1, Figure 5, §8, Finding 4 [Bản đã dùng].

Measuring Faithfulness in CoT Reasoning [11]. *Công trình nền tảng.*
Measuring Faithfulness in Chain-of-Thought Reasoning.
Bài toán. Kiểm tra đáp án có phụ thuộc thực chất vào CoT được viết ra hay không.
Phương pháp. Thực hiện các can thiệp phản thực nghiệm (counterfactual perturbations) trên chuỗi CoT của các mô hình lên tới 175B tham số: trả lời sớm, chèn lỗi bước trung gian, diễn đạt lại, và thay thế CoT bằng filler tokens "...". Các phép truncation, chèn lỗi và filler tạo can thiệp trên vết suy luận. Chúng kiểm tra mức phụ thuộc của đáp án vào trace, không chỉ độ thuyết phục mà người đọc cảm nhận.

Thiết lập và metric. Các model pre-o1 đến 175B được thử bằng early answering, chèn lỗi, paraphrase và filler. Metric theo thay đổi đáp án/task accuracy sau perturbation; không phải benchmark cho mọi kiến trúc reasoning mới.
Kết quả. Với các filler tokens, mô hình thử nghiệm không cải thiện độ chính xác so với direct answer; tác giả nêu giả thuyết hiệu ứng lệch phân phối (OOD) như một khả năng giải thích hiệu năng giảm, không phải nguyên nhân đã đo đạc độc lập.
Đóng góp. Cung cấp phương pháp can thiệp nền tảng cho đánh giá faithfulness.
Hạn chế. Kết quả thực hiện trên các mô hình pre-o1 và prompt-based CoT; không tự động khái quát thành quy luật phủ định cho mọi nghiên cứu filler-token hay latent reasoning sau này. Scope foundational.
Nhận xét đối chiếu. Dùng làm nền tảng thiết kế thí nghiệm faithfulness. Can thiệp có thể đưa model ra ngoài phân phối; cần matched controls. Kết quả ở model pre-o1 không thay thế kiểm chứng trên reasoning model và kiến trúc latent sau này.
Vị trí nguồn: arXiv:2307.13702v1, §2.1, §2.3, §2.5.1, Figure 5, Table 2 [Bản đã dùng].

A3.2 — Redundant steps và cấu trúc graph

Trích topology, dự đoán correctness và phân tích nhánh bị bỏ dở hoặc dependency không được đáp ứng.
Cơ chế và cách tiếp cận. Graph cho phép mô tả branching, merging và dependency thay vì chỉ length tuyến tính. CoT2Graph học predictor từ topology; FSF đo phần reasoning thuộc các nhánh bị bỏ dở. Chúng cung cấp tín hiệu vượt ngoài số token, nhưng có đối tượng dự đoán và cách dùng khác nhau.
Đối chiếu và ranh giới. Cần tách ba kết quả: feature dự đoán correctness, feature chọn được đáp án tốt hơn, và can thiệp loại feature làm continuation tốt hơn. Graph extraction và judge là thành phần của phép đo; GNN attribution không trực tiếp giải thích causal mechanism của generator.

CoT2Graph [2]. *Long-CoT trực tiếp.*
Understanding Failures in LLM Reasoning by Learning Structured Representations of Chain-of-Thought.
Bài toán. Phân tích failure mode mà feature độ dài tuyến tính bỏ sót.
Phương pháp. Chuyển đổi vết CoT thành đồ thị phụ thuộc định kiểu nắm bắt phân nhánh và gộp bài toán con; huấn luyện Graph Neural Networks (GNN) kết hợp GNNExplainer để trích xuất đặc trưng cấu trúc liên quan đến độ đúng đắn và dư thừa. GNN nhận đồ thị được trích từ trace rồi dự đoán tính đúng/sai; GNNExplainer phân tích bộ dự đoán đó. Best-of-N selection là ứng dụng riêng của tín hiệu cấu trúc.
Thiết lập và metric. Phân biệt hai setup: dự đoán correctness trên GPQA/DS-Qw14B và Best-of-5 selection trên 600 câu MMLU-Pro/DS-Qw7B. Baseline prediction gồm length-based Logistic Regression và LCoT2Tree; selection có ORM và uncertainty.
Kết quả. Trên GPQA với vết suy luận DS-Qw14B, độ chính xác dự đoán đúng/sai của đáp án cuối đạt 65.4%, tăng +7.8 điểm phần trăm (pp) so với baseline Logistic Regression theo độ dài (57.6%) và +2.5 pp so với baseline cây LCoT2Tree (62.9%); trên 600 bài MMLU-Pro (N=5), CoT2Graph đạt 54.5% và graph+entropy đạt 56.1%, so với ORM-Best (50.8%), ORM+uncertainty (51.6%) và mean (43.2%); phân tích GNNExplainer giải thích bộ dự đoán GNN chứ không phải cơ chế nhân quả nội tại của LLM.
Đóng góp. Cho thấy topology mang tín hiệu dự đoán hữu ích và có thể hỗ trợ selection.
Hạn chế. Cấu trúc đồ thị phản ánh tốt tính dư thừa và kế hoạch nhưng ít mang tín hiệu về lỗi hallucination nội bộ bước; phụ thuộc vào parser trích xuất đồ thị phụ thuộc.
Nhận xét đối chiếu. Dùng để đánh giá lợi ích của topology so với length-only features. Phải phân biệt accuracy của correctness predictor với accuracy giải bài sau selection; chi phí parser/GNN/judge và khả năng đổi model/domain là phần cần kiểm tra.

Vị trí nguồn: OpenReview forum:wFTJZ8fvns, §4.2, Table 1, §5.6, Figure 7 [Bản đã dùng].

What Characterizes Effective Reasoning? [7]. *Long-CoT trực tiếp.*
What Characterizes Effective Reasoning? Revisiting Length, Review, and Structure of CoT.
Bài toán. Xác định cấu trúc trace liên quan reasoning hiệu quả hơn length/review.
Phương pháp. Trích đồ thị, đo Failed-Step Fraction (FSF), chọn ứng viên và can thiệp loại nhánh bỏ dở. FSF là tỷ lệ nút thuộc các nhánh bị bỏ dở trong graph do judge trích. Selection trên tập ứng viên cố định và continuation sau branch editing là hai thí nghiệm khác nhau.
Thiết lập và metric. Phân tích 10 LRM trên reasoning toán/khoa học; một setup giữ candidate pool để so selector, setup khác edit nhánh trong trace HARP sai của R1/GPT-oss-120B rồi đánh giá continuation. Hai kết quả không được gộp thành một phép đo.
Kết quả. Trên 10 LRM, FSF dự đoán correctness nhất quán hơn length/review; thử bỏ nhánh trên các trace HARP sai của R1 và GPT-oss-120B cải thiện khả năng tiếp tục giải đúng.
Đóng góp. Nối phân tích cấu trúc với selection và context control.
Hạn chế. FSF phụ thuộc judge; nhánh bỏ dở không đồng nghĩa bước sai logic.
Nhận xét đối chiếu. Dùng để nối phân tích cấu trúc với kiểm soát context. Cần tính chi phí sinh ứng viên và graph judge; bỏ nhánh thành công trong thiết lập này không chứng minh mọi exploration thất bại đều vô ích đối với học hoặc kiểm chứng.
Vị trí nguồn: v1, §3.2, §4–5, Table 1 [Bản đã dùng].

A3.3 — Complexity, error propagation và giới hạn horizon

Phân tích suy giảm theo độ khó hoặc chuỗi phụ thuộc; không đồng nhất complexity với horizon.
Cơ chế và cách tiếp cận. Một trục độ khó là search-space complexity như ZebraLogic; trục khác là chuỗi dependency dài như phân tích DAG của LongCoT. Accuracy suy giảm theo hai trục có thể giống nhau ở bề mặt nhưng chưa chứng minh cùng nguyên nhân.
Đối chiếu và ranh giới. Error accumulation cần giả định rõ nếu dùng baseline độc lập p^N. Suy giảm nhanh hơn baseline là động lực phân tích state carrying hoặc tương tác lỗi, chưa xác định duy nhất attention, memory hay nội dung trace là cơ chế gây ra.

ZebraLogic [16]. *Nghiên cứu suy luận liên quan.*
ZebraLogic: On the Scaling Limits of LLMs for Logical Reasoning.
Bài toán. Đo giới hạn reasoning khi độ phức tạp tổ hợp tăng.
Phương pháp. Thiết kế khuôn khổ câu đố lưới logic ZebraLogic dựa trên bài toán CSP với không gian tìm kiếm định lượng chính xác (<10^3 đến >10^9 ứng viên); đánh giá tác động của scaling tham số và scaling tính toán suy luận (Best-of-N, Self-Refinement). Độ phức tạp được thao tác qua câu đố CSP, giúp khảo sát scaling thay vì chỉ nhóm câu hỏi theo độ dài output. Kích thước search space và độ dài vết giải không phải cùng đại lượng.
Thiết lập và metric. Câu đố lưới CSP có mức search-space complexity; chấm accuracy theo tổng thể và các mức như Large/X-Large. So model size và các chiến lược test-time trong cùng họ bài toán.
Kết quả. Ghi nhận hiện tượng Lời nguyền phức tạp (Curse of Complexity): độ chính xác giảm sút nghiêm trọng khi không gian tìm kiếm vượt 10^6; Llama-3.1-405B đạt 32.6% tổng thể nhưng giảm về 1.5% ở mức Large và 0.0% ở X-Large; reasoning models (o1-full 42.5%, DeepSeek-R1 28.5% ở X-Large) cũng suy giảm mạnh; scaling test-time chỉ cải thiện cận biên trên các mô hình được thử nghiệm.
Đóng góp. Định lượng suy giảm theo complexity, bổ sung cho nghiên cứu long-horizon failure.
Hạn chế. Khảo sát trên dạng bài toán câu đố lưới CSP cụ thể; không trực tiếp nghiên cứu vết suy luận mở rộng dạng tự do (scope adjacent_reasoning, primary A3, secondary A4).
Nhận xét đối chiếu. Dùng làm adjacent evidence cho scaling limits, không là bằng chứng riêng rằng Long-CoT vô hiệu. Kết luận nên gắn với puzzle family và mô hình đã thử; cần tách combinatorial difficulty khỏi tải ghi nhớ trạng thái.

Vị trí nguồn: arXiv:2502.01100v2, §3.1, Table 1, §3.2, §5, Figure 6 [Bản đã dùng].

A3.4 — Reasoning behavior: overthinking/underthinking

Khảo sát lặp thừa, đổi hướng sớm và self-doubt; phân biệt tổng độ dài với độ sâu exploration.
Cơ chế và cách tiếp cận. Overthinking là compute thừa, underthinking là thiếu phát triển đủ sâu một hướng, self-doubt là lặp kiểm tra điều đã có. Một trace có thể vừa dài vừa nông vì đổi hướng liên tục; các nhãn không loại trừ nhau.
Đối chiếu và ranh giới. Do đó không rút kết luận chung rằng reflection luôn hữu ích hoặc luôn dư thừa. Cần ghi loại bài, thời điểm can thiệp và tác động lên độ đúng; giảm token, tăng độ sâu và giảm switching không phải cùng mục tiêu.

Do NOT Think That Much for 2+3=? [5]. *Long-CoT trực tiếp.*
Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs.
Bài toán. Định lượng compute thừa sau khi đã tìm được lời giải.
Phương pháp. Chỉ số outcome/process efficiency; self-training với preference cho lời giải ngắn và giữ phản tư. Phân biệt efficiency theo outcome với phần tiến trình thực sự đóng góp; self-training chọn trace ngắn rồi vẫn giữ reflection để tránh loại cả suy nghĩ hữu ích.
Thiết lập và metric. Trên QwQ-32B-Preview, so các cách self-training/preference optimization và loại trace được giữ. Kết quả MATH500 dùng SimPO với First-Correct Solution + Reflection; token và accuracy được báo đồng thời.
Kết quả. QwQ-32B-Preview, MATH500: SimPO FCS+Reflection giảm 2407.9 xuống 1330.7 token; accuracy 93.0% xuống 92.8%.
Đóng góp. Kết nối chẩn đoán overthinking với lựa chọn dữ liệu và preference training.
Hạn chế. Giảm token vẫn có thể mất accuracy, nhất là bài khó.
Nhận xét đối chiếu. Dùng để giải thích vì sao shorter không tự động better. MATH500 tiết kiệm token lớn nhưng accuracy giảm nhẹ; chưa thể gọi là tương đương thống kê nếu thiếu uncertainty. First-correct solution trong construction dùng đáp án đúng, không phải oracle sẵn có lúc triển khai.
Vị trí nguồn: v2, §2–3, Table 4 [Bản đã dùng].

Thoughts Are All Over the Place [24]. *Long-CoT trực tiếp.*
Thoughts Are All Over the Place: On the Underthinking of o1-Like LLMs.
Bài toán. Giải thích trace dài nhưng đổi hướng trước khi khám phá đủ sâu.
Phương pháp. Underthinking score và TIP: phạt logits chuyển hướng trong một cửa sổ giải mã. Underthinking nói về thiếu độ sâu trên một hướng, không nhất thiết tổng trace ngắn. TIP chỉ điều chỉnh xác suất chuyển hướng trong một cửa sổ, không có verifier xác nhận hướng hiện tại đúng.
Thiết lập và metric. TIP dùng penalty strength/duration chọn trên AIME22–23. Với QwQ-32B-Preview ở AIME24, Pass@1 được ước lượng từ 32 mẫu mỗi bài; không so trực tiếp với con số greedy từ setup khác.
Kết quả. QwQ-32B-Preview, AIME24: Pass@1 tăng 38.3% lên 44.1%, ước lượng từ 32 mẫu/bài; alpha=3, beta=600, chọn trên AIME22–23.
Đóng góp. Phân biệt độ dài tổng với độ sâu phát triển từng hướng suy luận.
Hạn chế. Phụ thuộc token chuyển hướng và siêu tham số; không mặc nhiên tiết kiệm compute.
Nhận xét đối chiếu. Dùng làm phản biện việc luôn khuyến khích suy nghĩ ngắn hơn. Kiểm tra cùng sampling và budget; penalty quá mạnh có thể giữ model trong nhánh sai. Không so trực tiếp số greedy ở bảng khác với Pass@1 ước lượng từ nhiều mẫu.
Vị trí nguồn: v2, §2–3, Tables 2–3 [Bản đã dùng].

Revisiting Overthinking from Self-Doubt [20]. *Long-CoT trực tiếp.*
Revisiting Overthinking in Long Chain-of-Thought from the Perspective of Self-Doubt.

Bài toán. Khảo sát verification lặp lại và hành vi cố giải đề thiếu tiền đề.
Phương pháp. Phân tích hành vi suy luận dưới góc nhìn tự nghi ngờ (self-doubt) — xu hướng kiểm tra lại liên tục các kết luận trung gian đã đúng; khảo sát trên 4 mô hình với GSM8K, MATH-500 và bộ đề thiếu tiền đề (Missing Premise); đề xuất prompt can thiệp kiểm tra tiền đề. Self-doubt được xem ở mức hành vi: kiểm lại kết luận đã có. Bộ đề thiếu tiền đề đặt thêm câu hỏi liệu mô hình có nhận ra bài không đủ điều kiện để giải.
Thiết lập và metric. Khảo sát self-doubt trên bốn model với GSM8K, MATH-500 và bộ Missing Premise. Ở đề thiếu tiền đề, cần đo cả token và khả năng nhận ra bài không đủ điều kiện để giải.
Kết quả. Self-doubt chiếm tỷ lệ lớn trong các bước overthinking thừa thãi; prompt can thiệp kiểm tra tiền đề giúp giảm 35%–45% số token trong khi bảo toàn hoặc tăng nhẹ độ chính xác trên mô hình thử nghiệm; trên bài thiếu tiền đề, tỷ lệ từ chối giải đúng tăng từ 30-50% lên 75-97%.
Đóng góp. Làm rõ một dạng reasoning behavior gây lãng phí và cách can thiệp đơn giản.
Hạn chế. Can thiệp dựa trên prompt mang tính heuristic; chưa giải quyết triệt để overthinking bắt nguồn từ cơ chế tối ưu hóa RL nội tại.
Nhận xét đối chiếu. Dùng để tách unnecessary verification khỏi productive reflection. Prompt kiểm tra tiền đề có thể đổi chiến lược trả lời hoặc từ chối; tiết kiệm token trên dạng đề này chưa phải cải thiện chung cho mọi bài toán có đáp án.
Vị trí nguồn: arXiv:2505.23480v1, §2, Figure 1, §5, Table 1, Table 3, §6 [Bản đã dùng].

Bảng đối chiếu trong A3. *Nhận xét tổng hợp, không phải xếp hạng có kiểm soát.*

Hướng/paper	Thành phần hoặc đơn vị chính	Điểm cần đọc kèm
Discourse, faithfulness	ReasoningFlow, Lanham	Schema hay perturbation hỗ trợ claim nào; có distribution shift không?
Graph/failure	CoT2Graph, FSF	Đang đo predictor, selector hay hiệu quả can thiệp prefix?
Complexity, horizon	ZebraLogic, dẫn chéo LongCoT	Trục bị thao tác là search space hay số dependency?
Behavior	Chen, Wang, Peng	Lặp thừa, đổi hướng sớm và self-doubt có thể cùng xảy ra.

Tổng hợp hướng nghiên cứu và hạn chế chung. Nhiều token không bảo đảm exploration đủ sâu, và verification lặp lại không nhất thiết có ích. Các nghiên cứu graph bổ sung tín hiệu ngoài length, còn benchmark complexity/horizon cho thấy giới hạn của scaling. Cần tách mô tả tương quan, correctness prediction, trace intervention và internal intervention; một feature dự đoán tốt chưa phải cơ chế nhân quả được chứng minh.

Liên hệ dependency khi có bằng chứng. Discourse/structural graph có thể dùng để mô tả hoặc đánh giá dependency; chỉ ghi loại bằng chứng tương ứng. ReasoningFlow không cho phép suy từ cạnh discourse sang causal edge nội tại; ZebraLogic không cô lập riêng error propagation trong Long-CoT. B=N/A khi paper chỉ mô tả hành vi mà chưa đáp ứng tiêu chí chức năng dependency.

1.4 A4: Evaluation/Benchmark

Đo năng lực reasoning dài, process quality và hiệu quả compute bằng những bài toán và metric nào?

A4.1 — Benchmark long-horizon và planning

Tăng dependency horizon/độ khó và kiểm tra state hoặc ràng buộc của lời giải.
Cơ chế và cách tiếp cận. Benchmark long-horizon cần làm rõ horizon được xác định bằng dependency, số bước hợp lệ hay độ dài solution. LongCoT dùng template nhiều miền và phân tích DAG; SPaRC kiểm tra planning trong lưới có luật rõ.
Đối chiếu và ranh giới. Chỉ số accuracy cuối nên đi cùng lỗi state/constraint và mức khó. Không dùng output length làm đại diện duy nhất cho logical depth; cũng không so human/model time mà bỏ qua khác biệt về protocol và biểu diễn input.

LongCoT benchmark [18]. *Long-CoT trực tiếp.*
LongCoT: Benchmarking Long-Horizon Chain-of-Thought Reasoning.
Bài toán. Đánh giá reasoning qua nhiều dependency và horizon dài.
Phương pháp. Xây dựng benchmark LongCoT gồm 2,500 bài toán dựa trên template tham số hóa do chuyên gia thiết kế trên 5 miền (Toán, Hóa, CS, Cờ vua, Logic); tiến hành phân tích riêng rẽ thực nghiệm DAG phụ thuộc tuần tự (1 đến 35 nút từ Omni-Math) và bộ benchmark tổng hợp; đánh giá frontier models và scaffolding RLM. Tách bộ benchmark template đa miền khỏi thí nghiệm DAG tăng số dependency. Độ dài solution của benchmark và số nút DAG là hai cách thao tác horizon khác nhau.
Thiết lập và metric. Benchmark tổng có 2,500 bài thuộc năm miền; điểm model nêu ở đây dùng 2,000 bài vừa/khó. Thí nghiệm DAG từ 1 đến 35 node là một phân tích riêng về dependency horizon.
Kết quả. Trên 2,000 bài vừa và khó, các frontier models được thử nghiệm đạt độ chính xác khiêm tốn (GPT 5.2 đạt 9.83%, Gemini 3 Pro 6.08%, Grok 4.1 2.04%); trong thực nghiệm DAG phụ thuộc, độ chính xác sụt giảm nhanh hơn đường cơ sở lỗi độc lập lý thuyết (p^N), đặc biệt sau 15 nút; scaffolding RLM thuần suy luận không cải thiện hiệu năng.
Đóng góp. Cung cấp stress test trực tiếp cho long-horizon reasoning và state carrying.
Hạn chế. Hiện tượng suy giảm được tác giả giải thích như giả thuyết về mất mát trạng thái ngữ cảnh, chưa phải can thiệp nhân quả đo đạc trực tiếp cơ chế attention nội tại.
Nhận xét đối chiếu. Dùng để stress-test state carrying. Accuracy thấp trên bộ vừa/khó không đại diện cho mọi reasoning task; đường p^N chỉ là baseline độc lập giản lược, chưa định danh duy nhất nguyên nhân suy giảm là attention hay memory loss.
Vị trí nguồn: arXiv:2604.14140v1, §3, §4.1, Figure 4, §4.2, Figure 6 [Bản đã dùng].

SPaRC [9]. *Long-CoT trực tiếp.*
SPaRC: A Spatial Pathfinding Reasoning Challenge.
Bài toán. Đo planning nhiều bước, giữ ràng buộc và thích ứng compute theo độ khó.
Phương pháp. Giới thiệu SPaRC gồm 1,000 bài toán tìm đường lưới 2D chia theo 5 cấp độ khó; đánh giá 3 mô hình instruct, 4 reasoning models (o4-mini, o3-mini, QwQ-32B, R1-Distill-70B) và 6 người giải. Bài toán tìm đường cho phép kiểm tra tính hợp lệ của từng move, không chỉ chuỗi mô tả nghe hợp lý. Phân tầng độ khó hỗ trợ phân tích compute adaptation.
Thiết lập và metric. 1,000 bài tìm đường lưới 2D, năm difficulty tier; so instruct model, reasoning model và người giải. Báo độ đúng đường đi, vi phạm luật và lượng reasoning token theo độ khó.
Kết quả. Người giải đạt 98.0% tổng thể và 94.5% ở Level 5; mô hình reasoning cao nhất (o4-mini) đạt 15.8% tổng thể và 1.1% ở Level 5 (trên 50% đường đi vi phạm luật); trong khi người giải tăng thời gian suy nghĩ theo độ khó, mô hình thử nghiệm (như QwQ-32B) phát ra lượng token tương đương giữa các mức khó (14,433 tokens mức 1 vs 13,114 tokens mức 5).
Đóng góp. Bổ sung đánh giá planning/state constraints ngoài miền toán.
Hạn chế. Giới hạn trong miền tìm đường lưới 2D biểu diễn bằng văn bản; kết luận về phân bổ compute áp dụng cho các mô hình và cấu hình prompt được khảo sát.

Nhận xét đối chiếu. Dùng để bổ sung planning/state constraints ngoài toán. Phải giữ cách mã hóa grid, luật di chuyển và prompt; số token gần như không đổi là quan sát về allocation, chưa đủ để giải thích vì sao mô hình thất bại.
Vị trí nguồn: arXiv:2505.16686v2, §3, Figure 2, Table 1, Table 2 [Bản đã dùng].

A4.2 — Metric chất lượng, độ dài và redundancy

Bổ sung chỉ số ở cấp graph/step cho accuracy, token cost và latency.
Cơ chế và cách tiếp cận. Chất lượng reasoning có ít nhất outcome correctness, process quality và hiệu quả sử dụng tài nguyên. CoTJudger bổ sung metric redundancy ở cấp graph; FSF và outcome/process efficiency của các paper A3 cung cấp các góc đo liên quan.
Đối chiếu và ranh giới. Token, node, step và wall-clock time không hoán đổi được. Một bước ngoài SEP vẫn có thể hữu ích trong quá trình tìm lời giải; metric structural redundancy cần kiểm chứng khi dùng để quyết định bỏ bước hoặc so các model có segmentation khác nhau.

CoTJudger [14]. *Long-CoT trực tiếp.*
CoTJudger: A Graph-Driven Framework for Automatic Evaluation of Chain-of-Thought Efficiency and Redundancy in LRMs.
Bài toán. Đo redundancy cấu trúc thay vì chỉ đáp án hoặc tổng token.
Phương pháp. Khung CoTJudger phân đoạn vết giải, phân loại nút nguyên tử, xây dựng CoT Graph, và trích xuất Shortest Effective Path (SEP) bằng DFS forward kết hợp kiểm tra tính nhất quán qua LLM; định nghĩa Redundancy Ratio R=(|V|-L_eff)/|V|. Đơn vị của redundancy ratio là nút/step sau segmentation, không phải token. SEP được tìm trên graph đã trích và có kiểm tra nhất quán; preprocessing thay đổi có thể thay đổi mẫu số lẫn đường hiệu quả.
Thiết lập và metric. Đánh giá 21 reasoning model sau segmentation, phân loại node và trích graph/SEP. Đơn vị của redundancy ratio là node/step, không phải token; kết quả phụ thuộc cả graph extraction và consistency judge.
Kết quả. Đánh giá trên 21 reasoning models ghi nhận tỷ lệ đáng kể các nút/bước suy luận nằm ngoài đường dẫn hiệu quả ngắn nhất SEP; phần lớn bước nằm ngoài SEP tập trung ở các vòng lặp tự kiểm tra sau khi đã đạt trạng thái có thể kết luận; mô hình chưng cất thể hiện mối tương quan về độ dài vết giải tương tự giáo viên nhưng mật độ nút hiệu quả trên bước suy luận thấp hơn.
Đóng góp. Cung cấp metric tự động cho efficiency ở cấp node/step.
Hạn chế. SEP là một structural proxy phụ thuộc vào mô hình thẩm định (judge-dependent) chứ không chứng minh mọi bước bị loại bỏ đều không cần thiết về mặt ngữ nghĩa; quan hệ giáo viên - học sinh là tương quan quan sát được, không phải bằng chứng nhân quả về sự kế thừa.
Nhận xét đối chiếu. Dùng làm structural efficiency metric, bổ sung cho accuracy và cost. Một bước ngoài SEP có thể là exploratory work cần thiết để phát hiện lời giải; muốn khẳng định có thể bỏ an toàn cần ablation tiếp tục sinh hoặc đánh giá lại đáp án.
Vị trí nguồn: arXiv:2603.07078v1, §4.1–4.6, Figure 2, §5, Figure 3 [Bản đã dùng].

A4.3 — Process/outcome evaluation và verifier

Đo tìm lỗi, nhận diện lời giải đúng và tách generation, critique, correction.
Cơ chế và cách tiếp cận. ProcessBench kiểm tra error localization/no-error; CriticBench tách generation, critique và correction. PRM, general LLM critic và reasoning model critic là các evaluator cần được kiểm tra trước khi tích hợp vào training hoặc search.
Đối chiếu và ranh giới. Tìm lỗi đầu tiên đúng chưa đồng nghĩa viết được feedback sửa lỗi hoặc chọn đúng branch. Báo riêng chất lượng verifier, correction sau feedback và outcome cuối; giữ threshold selection và task difficulty để tránh so metric không cùng giao thức.

ProcessBench [32]. *Nghiên cứu suy luận liên quan.*

ProcessBench: Identifying Process Errors in Mathematical Reasoning.
Bài toán. Đánh giá khả năng tìm bước lỗi đầu tiên hoặc nhận ra lời giải không lỗi.
Phương pháp. Xây dựng ProcessBench gồm 3,400 trường hợp kiểm thử (test cases) kèm lời giải từng bước từ 4 tập dữ liệu toán (GSM8K, MATH, OlympiadBench, Omni-MATH); chuyên gia gán nhãn xác định bước lỗi sớm nhất; đánh giá PRMs và LLM Critics qua điểm F1 trung bình trên 4 tập con. Nhiệm vụ là xác định bước lỗi sớm nhất hoặc nhận ra lời giải không lỗi, không phải viết lời giải mới. F1 ở đây là harmonic mean giữa accuracy trên mẫu có lỗi và mẫu đúng (§4.1).
Thiết lập và metric. 3,400 mẫu trên GSM8K, MATH, OlympiadBench và Omni-MATH. Nhiệm vụ là tìm lỗi đầu tiên hoặc no-error; F1 riêng là harmonic mean accuracy trên mẫu có lỗi và mẫu đúng, rồi báo trung bình theo subset.
Kết quả. Skywork-PRM-7B đạt điểm F1 trung bình 42.1% trên 4 tập con; các reasoning models đóng vai trò critic đạt F1 trung bình: QwQ-32B-Preview đạt 71.5%, GPT-4o đạt 61.9%, và o1-mini đạt 87.9%.
Đóng góp. Cung cấp benchmark process evaluation và kiểm tra generalization của verifier/critic.
Hạn chế. Giới hạn trong suy luận toán học với độ khó hỗn hợp; bài toán định vị bước lỗi đầu tiên chưa mô tả toàn bộ cấu trúc lỗi của vết giải nhiều nhánh. Phạm vi adjacent_reasoning.
Nhận xét đối chiếu. Dùng để kiểm định verifier/critic trước khi dùng cho A1/A2. Điểm trung bình qua bốn subset không phải một accuracy gộp; cần báo riêng miền khó và quy tắc chọn threshold. Phát hiện lỗi đúng chưa bảo đảm sửa được lỗi.
Vị trí nguồn: arXiv:2412.06559v4, §3.1, §4.2, Table 2, Table 3 [Bản đã dùng].

CriticBench [17]. *Nghiên cứu suy luận liên quan.*
CriticBench: Benchmarking LLMs for Critique-Correct Reasoning.
Bài toán. Tách năng lực giải, critique và correction.
Phương pháp. CriticBench bao phủ 5 miền suy luận (Toán, Code, Thuật toán, Logic đời thường, Suy luận ký hiệu) tích hợp từ 15 datasets; đánh giá 17 LLMs qua 3 giai đoạn: tự sinh (G), phê phán (Q), và sửa sai dựa trên critique (C). Thiết kế Generation–Critique–Correction tách năng lực giải ban đầu, nhận xét và sửa sau phản hồi. Đây là đối chiếu cho self-correction, không phải benchmark chuyên cho horizon rất dài.
Thiết lập và metric. Đánh giá 17 LLM qua năm miền theo Generation–Critique–Correction. Cần giữ riêng năng lực giải ban đầu, độ đúng critique và kết quả correction sau feedback; đây không phải metric horizon chuyên biệt.
Kết quả. Ghi nhận mối tương quan giữa độ chính xác phê phán và tỷ lệ sửa sai thành công; các bài toán lập trình và thuật toán có tỷ lệ sửa sai cao hơn nhờ phản hồi rõ ràng; các mô hình nhỏ khi tự phê phán có xu hướng bảo thủ nếu thiếu verifier ngoài.
Đóng góp. Cung cấp đối chiếu process/outcome và hiệu quả feedback cho reasoning.
Hạn chế. Đánh giá trên các prompt-based LLMs thế hệ trước; chưa chuyên biệt hóa cho các vết Long-CoT hàng vạn token của reasoning models mới (scope adjacent_reasoning).
Nhận xét đối chiếu. Dùng để đánh giá chất lượng phản hồi thay vì chỉ đếm reflection tokens. Tương quan critique–correction không cô lập tác động nhân quả; cần giữ cùng lời giải đầu vào, feedback source và compute khi so critic.

Vị trí nguồn: arXiv:2402.14809v4, §3, §4.1, Table 1, Figure 6 [Bản đã dùng].

Bảng đối chiếu trong A4. *Nhận xét tổng hợp, không phải xếp hạng có kiểm soát.*

Hướng/paper	**Thành phần hoặc đơn vị chính**	**Điểm cần đọc kèm**
LongCoT	Bài toán và dependency chain	Outcome accuracy theo horizon; tách dataset tổng và DAG test.
SPaRC	Đường đi và từng move	Validity/ràng buộc, difficulty tier, token allocation.
CoTJudger	Node/step và SEP	Redundancy cấu trúc; không quy trực tiếp ra token cost.
ProcessBench	Lời giải có hoặc không có lỗi	Earliest-error/no-error accuracy và F1 theo giao thức riêng.
CriticBench	Generation–critique–correction	Phân biệt chất lượng feedback với thành công sửa đáp án.

Tổng hợp hướng nghiên cứu và hạn chế chung. Outcome accuracy không thay thế process quality; error localization khác correction success; token count khác step redundancy. Dependency horizon cần được thao tác hoặc đo riêng thay vì suy từ output length. ProcessBench dùng F1 riêng theo harmonic mean accuracy trên mẫu lỗi/mẫu đúng; metric graph còn phụ thuộc segmentation và judge.

Liên hệ dependency khi có bằng chứng. LongCoT tạo task-defined dependency để đánh giá state carrying; CoTJudger và ProcessBench liên quan assessment ở mức cấu trúc hoặc bước. Benchmark/metric đo được failure chưa có nghĩa đã đề xuất phương pháp repair hay chứng minh mechanism nội tại.

2 Related Work: phạm vi và cách phân loại của các survey

Phần này đối chiếu chín survey theo phạm vi, đơn vị phân loại, cách sử dụng trong bản review và giới hạn khi diễn giải taxonomy. Các survey không nằm trong 24 paper nghiên cứu A1–A4. Nguồn được ghim theo phiên bản đã đọc; phạm vi của bản mới hoặc bản xuất bản có thể thay đổi.

2.1 Các survey tiêu biểu

S1 — Chen và cộng sự — Towards Reasoning Era [4]. Phạm vi. Long-CoT trong reasoning LLM và khác biệt với short CoT.
Taxonomy chính. Deep Reasoning, Feasible Reflection và Extensive Exploration; các nhánh con gồm format/learning, feedback/refinement và internal/external exploration.
Diễn giải đơn vị phân loại. Các năng lực deep reasoning, feasible reflection và extensive exploration, không phải nhãn đóng góp của một paper. Nhánh feedback/refinement giúp tách phát hiện lỗi khỏi thực hiện sửa lỗi; format/learning giúp tách biểu diễn trace khỏi cách học.
Cách dùng cho Related Work. Dùng S1 để giới thiệu Long-CoT và giải thích vì sao reflection/exploration đáng được xem riêng. Sau đó dẫn cùng một phương pháp sang role thích hợp: phương pháp học phản tư thuộc A1, điều khiển phản tư thuộc A2, phân tích phản tư thuộc A3.
Ranh giới đối chiếu. Khung năng lực và khung research role trả lời hai câu hỏi khác nhau. Không nói S1 "thiếu A3/A4" chỉ vì tên nhánh chính không trùng taxonomy của nhóm.
Vị trí nguồn: Figures 2–3; §4–6 (v5).

S2 — Li và cộng sự — From System 1 to System 2 [15]. Phạm vi. Reasoning LLM rộng hơn riêng long trace.
Taxonomy chính. Features, Methods, Evolution; nhóm phương pháp gồm Structure Search, Reward Modeling, Self Improvement, Macro Action và Reinforced Fine-Tuning.
Diễn giải đơn vị phân loại. Features mô tả hành vi; methods tổ chức thuật toán; evolution mô tả tiến trình phát triển. Structure search, reward modeling, self-improvement, macro action và RFT cùng xuất hiện trong một bối cảnh reasoning LLM rộng.
Cách dùng cho Related Work. Dùng để giải thích nguồn gốc thuật toán và liên hệ giữa search với training. Một reward model có thể nằm trong training A1 hoặc làm evaluator/controller ở A2/A4; macro action liên quan cả học abstraction và thực thi subproblem.
Ranh giới đối chiếu. System 1–2 là khung tổ chức/diễn giải ở mức survey, không tự chứng minh LLM có cơ chế nhận thức giống người. Không đồng nhất mọi System-2 method với trace ngôn ngữ dài.
Vị trí nguồn: Figure 2; §3.1–3.3 (v6).

S3 — Zhang và cộng sự — A Survey on Test-Time Scaling [31]. Phạm vi. Tăng compute lúc inference, gồm cả sequential reasoning và parallel sampling.
Taxonomy chính. What, How, Where, How Well: dạng scaling, cách hiện thực, miền tác vụ và tiêu chí performance/efficiency/controllability/scalability.
Diễn giải đơn vị phân loại. What là dạng compute được mở rộng; How là phương pháp thực hiện; Where là miền tác vụ; How Well là tiêu chí đo kết quả. Một paper có thể nhận một giá trị ở mỗi trục, thay vì được đặt vào một ô duy nhất.
Cách dùng cho Related Work. Dùng để chuẩn hóa so sánh sequential Long-CoT, parallel sampling và hybrid search. A1/A2 phân biệt vị trí thay đổi training/inference; bốn W tiếp tục mô tả compute và miền đánh giá bên trong hai role đó.
Ranh giới đối chiếu. Test-time scaling bao gồm cả tăng số ứng viên mà mỗi trace không dài hơn. Số sample, độ dài mỗi sample và chi phí selection cần báo riêng.
Vị trí nguồn: Figure 2; §2–5, §8.2 (v3).

S4 — Wei và cộng sự — When LLM Meets Tree Search [25]. Phạm vi. Inference-as-search và các thiết kế tree search cho LLM.
Taxonomy chính. Node representation/granularity, evaluation function, MCTS dynamics và compute reporting; node có thể là trace, state–action hoặc terminal solution.
Diễn giải đơn vị phân loại. Representation/granularity của node, evaluation function, và các phép selection/expansion/simulation/backpropagation. Node là trace, state–action hay terminal solution làm thay đổi ý nghĩa của depth và rollout.
Cách dùng cho Related Work. Dùng cho A2.2 và đánh giá verifier trong search. Ví dụ MCTSr refine lời giải hoàn chỉnh nên không thể diễn giải một tầng cây như một bước lập luận nguyên tử của mọi phương pháp khác.
Ranh giới đối chiếu. Search bên ngoài và deliberation nội sinh đều có thể tiêu nhiều compute nhưng không có cùng đơn vị đo. Giá trị hữu ích của survey là làm rõ cơ chế search; không dùng tên taxonomy để giả định search tốt hơn flat Long-CoT.
Vị trí nguồn: §3.2–3.4 (v1).

S5 — Lee và Hockenmaier — Evaluating Step-by-step Reasoning Traces [12]. Phạm vi. Đánh giá reasoning trace và các bước trung gian.
Taxonomy chính. Tách tiêu chí factuality, validity, coherence, utility khỏi loại evaluator rule-based, intrinsic và external; có thảo luận meta-evaluation.
Diễn giải đơn vị phân loại. Tiêu chí cần đánh giá (factuality, validity, coherence, utility) tách khỏi cách đánh giá (rule-based matching, intrinsic metrics, external evaluators). Hai chiều này không ánh xạ một-một: cùng utility có thể được xấp xỉ bằng nhiều evaluator.
Cách dùng cho Related Work. Nguồn thuật ngữ cho A4 và phương pháp đọc A3. Khi paper báo correctness hay redundancy, cần chỉ rõ nó đang kiểm tra tiêu chí nào và bỏ ngỏ tiêu chí nào.
Ranh giới đối chiếu. Hữu ích về mặt diễn ngôn không đồng nghĩa faithful với tính toán nội tại. Dùng một LLM judge cũng tạo nhu cầu meta-evaluation và kiểm tra sai số judge; không biến output của judge thành ground truth tuyệt đối.
Vị trí nguồn: §3, §5 (v3).

S0 — Ji và cộng sự — A Survey of Test-Time Compute [8]. Phạm vi. Test-time adaptation và deliberate reasoning, không giới hạn explicit Long-CoT.
Taxonomy chính. System-1 adaptation theo model/input/representation/output; System-2 reasoning theo feedback, test-time strategies và improvement training.
Diễn giải đơn vị phân loại. Test-time adaptation cho System 1 và deliberate reasoning cho System 2. Nhánh reasoning gồm feedback modeling, test-time strategies và improvement training; nhánh adaptation còn bao gồm những thay đổi ngoài explicit CoT.
Cách dùng cho Related Work. Dùng ở đoạn mở rộng bối cảnh: Long-CoT là một cách sử dụng test-time compute, cùng tồn tại với adaptation và search. Khung feedback → strategy → training giúp nối A4, A2, A1.
Ranh giới đối chiếu. Không lấy toàn bộ literature test-time compute làm corpus Long-CoT trực tiếp. Paper nền/adjacent cần có nhãn phạm vi, như repo đang làm.
Vị trí nguồn: §3–4, §7 (v3).

S6 — Qu và cộng sự — A Survey of Efficient Reasoning for Large Reasoning Models [21]. Phạm vi. Efficient reasoning của LRM trong các giai đoạn phát triển và triển khai.
Taxonomy chính. Lifecycle gồm inference, SFT, RL, pretraining; inference chia length budgeting, system switch, model switch và parallel search.

Diễn giải đơn vị phân loại. Inference, SFT, RL, pretraining. Trong inference có length budgeting, system switch, model switch, parallel search; các phần training theo cơ chế nén hoặc học phân bổ.
Cách dùng cho Related Work. Dùng để so sánh efficiency đã học trong A1 với efficiency do controller ở A2, đồng thời đưa architectural cost vào thảo luận phạm vi.
Ranh giới đối chiếu. Ít token và ít compute không luôn đồng nghĩa, nhất là khi đổi model hoặc thêm routing/judge. Mở rộng về multimodal và ứng dụng phải phân biệt nội dung tổng hợp phương pháp với future directions.
Vị trí nguồn: Figure 3; §2–6, §7.5 (v2).

S7 — Sui và cộng sự — Stop Overthinking [22]. Phạm vi. Phương pháp giảm overthinking và tăng reasoning efficiency.
Taxonomy chính. Theo nơi can thiệp: model-based, reasoning-output-based, input-prompts-based; bổ sung dữ liệu, model compression và evaluation.
Diễn giải đơn vị phân loại. Model-based, reasoning-output-based, input-prompts-based; bổ sung data efficiency, model compression và evaluation. Đây là câu hỏi can thiệp vào đâu.
Cách dùng cho Related Work. Phân biệt tối ưu trọng số, sửa output/decoding và điều chỉnh prompt/routing. Cùng early stopping có thể nằm gần nhánh output; huấn luyện length reward gần nhánh model.
Ranh giới đối chiếu. Một pipeline có thể thay nhiều nơi cùng lúc nên cần multi-label khi đối chiếu. Không suy ra survey không có evaluation chỉ vì evaluation nằm ngoài ba nhánh phương pháp chính.
Vị trí nguồn: §3–7 (v4).

S8 — Zhu và Li — Towards Concise and Adaptive Thinking [33]. Phạm vi. Rút ngắn reasoning và thích ứng fast/slow thinking.
Taxonomy chính. Training-free gồm prompting, pipeline, decoding manipulation, merging; training-based gồm fine-tuning và RL.
Diễn giải đơn vị phân loại. Training-free gồm prompt, pipeline, decoding manipulation, merging; training-based gồm fine-tuning và RL. Đồng thời quan tâm lúc nào dùng fast/slow thinking.
Cách dùng cho Related Work. Nguồn gần trực tiếp cho A1.4 và A2.1/A2.4. Chi phí chuẩn bị của phương pháp và chi phí thực thi cần được đọc riêng.
Ranh giới đối chiếu. Training-free không đồng nghĩa không thêm module, không có calibration hay không tốn compute. Survey nêu loại trừ general compression, architectural optimization và multimodal efficiency; đó là lựa chọn phạm vi của tác giả, không phải thiếu sót được nhóm phát hiện.
Vị trí nguồn: §1, §3–4 (v1).

2.2 Đối chiếu các trục phân loại

Taxonomy theo capability, lifecycle, compute hoặc evaluator trả lời các câu hỏi khác nhau. Bảng dưới đây là crosswalk do nhóm tổng hợp, không phải điểm coverage hay khẳng định các survey dùng đúng nhãn A1–A4.

Mã	Trục tổ chức	Câu hỏi khi đưa vào A1–A4
S1	Năng lực/đặc trưng Long-CoT	Paper học, điều khiển, giải thích hay đo năng lực đó?
S2	Features / methods / evolution	Đóng góp trung tâm là algorithm, behavior hay evidence?
S3	What / How / Where / How Well	Tăng loại compute nào, ở đâu, với budget và metric nào?
S4	Node / evaluator / search dynamics	Đơn vị search và nơi feedback ảnh hưởng policy là gì?
S5	Tiêu chí và loại evaluator	Metric đo tiêu chí nào, có meta-evaluation không?
S0	Adaptation / deliberate reasoning	Long-CoT trực tiếp hay chỉ là bối cảnh compute?
S6	Lifecycle stage	Chi phí được chuyển từ inference sang training hay giảm toàn hệ thống?
S7	Đích can thiệp	Input, model, output hay nhiều thành phần cùng thay đổi?
S8	Training requirement	Thành phần nào học trước, thành phần nào thích ứng online?

2.3 Tổng hợp liên hệ với bản review A1–A4

Các survey trước tổ chức literature theo ít nhất sáu góc nhìn: *đặc trưng reasoning*, *tiến trình model*, *compute scaling*, *cấu trúc search*, *tiêu chí đánh giá* và *efficiency/adaptivity*. Bản review này giữ các góc nhìn đó để mô tả phương pháp, nhưng gán mỗi paper một primary role theo đóng góp trung tâm. Ví dụ s1 thuộc A1 cho dữ liệu/SFT và được dẫn chéo A2 cho budget forcing; CoTJudger thuộc A4 cho evaluator dù có phân tích graph liên quan A3.

Vì vậy, vai trò của Related Work là đặt cách tổ chức hiện tại vào bối cảnh literature đã có, không tuyên bố các survey trước thiếu toàn bộ training, inference, analysis hoặc evaluation. Giá trị tổng hợp được đề xuất là một danh mục có thể tra cứu theo A1–A4, các hướng nhỏ dựa trên phương pháp và tóm tắt từng paper theo bài toán, phương pháp, kết quả, đóng góp cùng hạn chế. Hồ sơ nguồn chi tiết được duy trì trong repo để các thành viên bổ sung và đối chiếu.

Tài liệu

[1] Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025. URL https://arxiv.org/abs/2502.04463v4.

[2] Tommaso Felice Banfi, Mubashara Akhtar, Jingwei Ni, and Mrinmaya Sachan. Understanding failures in LLM reasoning by learning structured representations of chain-of-thought. *OpenReview preprint forum:wFTJZ8fvns*, 2026. URL https://openreview.net/forum?id=wFTJZ8fvns. Author-reported Findings of EMNLP 2026.

[3] Kang Chen, Fan Yu, Junjie Nian, Shihan Zhao, Zhuoka Feng, Zijun Yao, Heng Wang, Minshen Yu, and Yixin Cao. Thinking traps in long chain-of-thought: A measurable study and trap-aware adaptive restart. *arXiv preprint arXiv:2601.11940*, 2026. URL https://arxiv.org/abs/2601.11940v1. Author-reported Findings of ACL 2026.

[4] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*, 2025. URL https://arxiv.org/abs/2503.09567v5.

[5] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do NOT think that much for 2+3=? on the overthinking of o1-like LLMs. *arXiv preprint arXiv:2412.21187*, 2024. URL https://arxiv.org/abs/2412.21187v2. Version 2, February 2025.

[6] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948v2.

[7] Yunzhen Feng, Julia Kempe, Cheng Zhang, Parag Jain, and Anthony Hartshorn. What characterizes effective reasoning? revisiting length, review, and structure of CoT. *arXiv preprint arXiv:2509.19284*, 2025. URL https://arxiv.org/abs/2509.19284v1.

[8] Yixin Ji, Juntao Li, Yang Xiang, Hai Ye, Kaixin Wu, Kai Yao, Jia Xu, Linjian Mo, and Min Zhang. A survey of test-time compute: From intuitive inference to deliberate reasoning. *arXiv preprint arXiv:2501.02497*, 2025. URL https://arxiv.org/abs/2501.02497v3.

[9] Lars Benedikt Kaesberg, Jan Philip Wahle, Terry Ruas, and Bela Gipp. SPaRC: A spatial pathfinding reasoning challenge. *arXiv preprint arXiv:2505.16686*, 2025. URL https://arxiv.org/abs/2505.16686v2. Author-reported EMNLP 2025.

[10] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025. URL https://arxiv.org/abs/2501.12599v4.

[11] Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukosiute, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023. URL https://arxiv.org/abs/2307.13702v1.

[12] Jinu Lee and Julia Hockenmaier. Evaluating step-by-step reasoning traces: A survey. *arXiv preprint arXiv:2502.12289*, 2025. URL https://arxiv.org/abs/2502.12289v3. Author-reported EMNLP 2025 Findings.

[13] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-T"ur, and Julia Hockenmaier. ReasoningFlow: Discourse structures for understanding LLM reasoning traces. *arXiv preprint arXiv:2606.05402*, 2026. URL https://arxiv.org/abs/2606.05402v2. Author-reported EMNLP 2026 Oral.

[14] Siyi Li, Jiajun Shi, Shiwen Ni, Ge Zhang, Shuaimin Li, Shijian Wang, Zhoufutu Wen, Yizhi Li, Hamid Alinejad-Rokny, Jiaheng Liu, Min Yang, and Wenhao Huang. CoT-Judger: A graph-driven framework for automatic evaluation of chain-of-thought efficiency and redundancy in LRMs. *arXiv preprint arXiv:2603.07078*, 2026. URL https://arxiv.org/abs/2603.07078v1. Author-reported ACL 2026 Main Conference.

[15] Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhiwei Li, Bao-Long Bi, Ling-Rui Mei, Junfeng Fang, Xiao Liang, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025. URL https://arxiv.org/abs/2502.17419v6.

[16] Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. ZebraLogic: On the scaling limits of LLMs for logical reasoning. *arXiv preprint arXiv:2502.01100*, 2025. URL https://arxiv.org/abs/2502.01100v2. Author-reported ICML 2025.

[17] Zicheng Lin, Zhibin Gou, Tian Liang, Ruilin Luo, Haowei Liu, and Yujiu Yang. CriticBench: Benchmarking LLMs for critique-correct reasoning. *arXiv preprint arXiv:2402.14809*, 2024. URL https://arxiv.org/abs/2402.14809v4. Author-reported Findings of ACL 2024.

[18] Sumeet Ramesh Motwani, Daniel Nichols, Charles London, Peggy Li, Fabio Pizzati, Acer Blake, Hasan Hammoud, Tavish McDonald, Akshat Naik, Alesia Ivanova, Vignesh Baskaran, Ivan Laptev, Ruben Glatt, Tal Ben-Nun, Philip Torr, Natasha Jaques, Ameya Prabhu, Brian Bartoldson, Bhavya Kailkhura, and Christian Schroeder de Witt. LongCoT: Benchmarking long-horizon chain-of-thought reasoning. *arXiv preprint arXiv:2604.14140*, 2026. URL https://arxiv.org/abs/2604.14140v1. Author-reported ICML 2026.

[19] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Fei-Fei Li, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. URL https://arxiv.org/abs/2501.19393v3.

[20] Keqin Peng, Liang Ding, Yuanxin Ouyang, Meng Fang, and Dacheng Tao. Revisiting overthinking in long chain-of-thought from the perspective of self-doubt. *arXiv preprint arXiv:2505.23480*, 2025. URL https://arxiv.org/abs/2505.23480v1.

[21] Xiaoye Qu, Yafu Li, Zhao-Chen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, Peng Li, Wei Wei, Jing Shao, Chaochao Lu, Yue Zhang, Xian-Sheng Hua, Bowen Zhou, and Yu Cheng. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond. *arXiv preprint arXiv:2503.21614*, 2025. URL https://arxiv.org/abs/2503.21614v2.

[22] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop

overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025. URL https://arxiv.org/abs/2503.16419v4.

[23] Renliang Sun, Wei Cheng, Dawei Li, Haifeng Chen, and Wei Wang. Stop when enough: Adaptive early-stopping for chain-of-thought reasoning. *arXiv preprint arXiv:2510.10103*, 2025. URL https://arxiv.org/abs/2510.10103v1.

[24] Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Linfeng Song, Dian Yu, Juntao Li, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Thoughts are all over the place: On the underthinking of o1-like LLMs. *arXiv preprint arXiv:2501.18585*, 2025. URL https://arxiv.org/abs/2501.18585v2.

[25] Jiaqi Wei, Xiang Zhang, Yuejin Yang, Wenxuan Huang, Juntai Cao, Sheng Xu, Xiang Zhuang, Zhangyang Gao, Muhammad Abdul-Mageed, Laks V. S. Lakshmanan, Chenyu You, Wanli Ouyang, and Siqi Sun. When LLM meets tree search: A systematic view of inference as search in large language models. *arXiv preprint arXiv:2608.30395*, 2026. URL https://arxiv.org/abs/2608.30395v1. Author-reported EMNLP 2026 Findings.

[26] Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Lifu Tang, Xiaowei Lv, Haosheng Zou, Yongchao Deng, Shousheng Jia, and Xiangzheng Zhang. Light-R1: Curriculum SFT, DPO and RL for long COT from scratch and beyond. *arXiv preprint arXiv:2503.10460*, 2025. URL https://arxiv.org/abs/2503.10460v4. Author-reported ACL 2025 Industry Track.

[27] Yang Xiang, Yixin Ji, Ruotao Xu, Dan Qiao, Zheming Yang, Juntao Li, and Min Zhang. When is thinking enough? early exit via sufficiency assessment for efficient reasoning. *arXiv preprint arXiv:2604.06787*, 2026. URL https://arxiv.org/abs/2604.06787v1. Author-reported ACL 2026 Main Conference.

[28] Chenxiao Yang, Nathan Srebro, and Zhiyuan Li. Recursive models for long-horizon reasoning. *arXiv preprint arXiv:2603.02112*, 2026. URL https://arxiv.org/abs/2603.02112v2. Author-reported ICML 2026.

[29] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. LIMO: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025. URL https://arxiv.org/abs/2502.03387v3.

[30] Di Zhang, Xiaoshui Huang, Dongzhan Zhou, Yuqiang Li, and Wanli Ouyang. Accessing GPT-4 level mathematical olympiad solutions via Monte Carlo tree self-refine with LLaMa-3 8b. *arXiv preprint arXiv:2406.07394*, 2024. URL https://arxiv.org/abs/2406.07394v2.

[31] Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. A survey on test-time scaling in large language models: What, how, where, and how well? *arXiv preprint arXiv:2503.24235*, 2025. URL https://arxiv.org/abs/2503.24235v3.

[32] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. ProcessBench: Identifying process errors in mathematical reasoning. *arXiv preprint arXiv:2412.06559*, 2025. URL https://arxiv.org/abs/2412.06559v4. Author-reported ACL 2025 Main Conference.

[33] Jason Zhu and Hongyu Li. Towards concise and adaptive thinking in large reasoning models: A survey. *arXiv preprint arXiv:2507.09662*, 2025. URL https://arxiv.org/abs/2507.09662v1.